

Ürün Mentaliteleri — Sandbox, Artifact ve Güvenlik

Kod çalıştıran her sistem aynı problemle karşılaşiyor: kod izole bir yerde çalışmalı ve o yer ölmeli, ama ürettiği kalmalı.

Bu belge on bir üründe bunun nasıl çözüldüğünü **akış olarak** anlatıyor, sonra ikisini karşılaştırıyor: **nicelik** (sayılar, sınırlar, ömürler) ve **güvenlik** (izolasyon, anahtar, patlama yarıçapı).

Tarih: 2026-09-08 · Kaynak: PTC_Piyasa_Mentaliteleri.md, PTC_Karsilastirma_Tablolari.md

Nasıl okunmalı

Her ürün bölümü aynı iskelette:

- Tek cümlelik tez** — bu ürün neye inanıyor
- Akış** — bir dosya nasıl yazılıyor ve nasıl geri okunuyor
- Bedeli** — bu tercihin ne kaybettiği

Sonda iki karşılaştırma tablosu var. Acele eden oraya bakabilir; ama asıl anlaşılması gereken şey **neden farklı seçtikleri**.

Bölüm 0 — Herkesin cevaplamak zorunda olduğu beş soru

Bütün karşılaştırmanın iskeleti bu. Her ürün, istese de istemese de bunları cevaplıyor:

#	Soru	Neden belirleyici
S 1	Kod nerede çalışıyor?	İzolasyonun gücü
S 2	Sandbox ne kadar yaşıyor?	Kalıcılığın nereden geleceği

S 3	Depoya nasıl erişiliyor?	Araya denetim koyabilir misin
S 4	Anahtar kimde?	Kod ele geçerse ne kaybedersin
S 5	Kayıt defteri var mı?	Dosya bir "artifact" mı yoksa sadece bayt mı

S3 ile S5 birbirine bağlı — ve bu, araştırmanın en keskin bulgusu:

Kayıt defteri, yalnızca yazma yolu **bir bileşenden geçtiğinde** ayakta kalıyor. Dosya sistemine mount edilen hiçbir üründe artifact registry yok.

Bölüm 1 — Ürünler tek tek

1.1 · Anthropic — "container'ı sakla"

Tez: Sandbox ölmesin; ölürse de checkpoint'ten diriltilsin.

Akış:

```
1 kod dosyayı $OUTPUT_DIR/ altına yazar
2 komut bitince platform DİZİNE BAKAR, yeni dosyayı görür
3 Files API'ye kaydeder, konuşmaya bir fiş döner: file_abc123
4 container ölür — bayt Files API'de kalır
5 sonraki tur: model konuşmada file_abc123'ü GÖRÜR
6 model ister → platform dosyayı container'a KOYAR
7 kod okur — sıradan bir dosya
```

Kilit fikir **vestiyer**: dosyanın kendisi bir yerde durur, ortalıkta sadece fişi dolaşır. 2 MB'lık bir Excel'i her mesajda taşımak yerine 12 karakterlik bir kimlik taşınıyor.

Ayrıca `container` id ile **container'ın kendisi** diriltilebiliyor: 5 dakikada bir checkpoint, 30 gün saklama. Yani değişkenler, kurulan paketler, RAM bile geri geliyor.

Ağ: "Completely disabled for security" — tamamen kapalı.

Bedeli: Kapalı platform. Soy ağacı yok, sürüm sabitleme yok, kayıt defterinde arama yok. Kendi bulutunuzda karşılığı yok.

1.2 · OpenAI — aynı fikir, daha kısa hafıza

Tez: Anthropic'inkiyle aynı desen; container + konuşma.

Akış: Aynı yedi adım. Kod `/mnt/data` altına yazar, platform hasat eder, container file content ucundan indirilir.

Fark: Kayıt defteri **yok** ve container **20 dakika hareketsizlikte** gidiyor. Anthropic'in 30 günü ile arasındaki fark üç büyüklük mertebesi.

Bedeli: Anthropic'inkilerin hepsi, üstüne bir de kimlik defteri yokluğu.

1.3 · Cloudflare — "kod yaz, tool çağırma"

Tez: LLM'e tool çağırtmak yerine kod yazdır (*Code Mode*). Depo zaten dosya sistemi.

Akış: Bucket (R2/S3/GCS) sandbox'a **mount** edilir. Kod `write()` der, baytlar doğrudan bucket'a gider. Arada bir katman yok.

İzolasyon: V8 isolate — JS motorunun içinde bir bölge. Dar ama sıkı.

Bedeli: Mount olduğu için **araya girecek yer yok**. Kayıt defteri yok, soy yok, TTL yok. Yazılan şeyin `artifact_id`'si yok — sadece bir S3 anahtarı.

1.4 · Google — üç ayrı ürün, üç ayrı cevap

Vertex AI Agent Engine: Adlandırılmış, uzun ömürlü sandbox — `sandbox_name` ile 14 gün yaşıyor.

GKE Agent Sandbox: Kubernetes-yerel, **gVisor** ile izole (araya sahte bir kernel giriyor). Ağ varsayılan reddet + allowlist.

ADK (Agent Development Kit): Bu üçü içinde bizi en çok ilgilendiren. `ArtifactService` diye gerçek bir API'si var: artifact adı + sürüm tutuyor. Ve `LoadArtifactsTool` şunu yapıyor:

İsimler her zaman prompt'ta, içerik talep üzerine, içerik geçmişe kalıcı yazılmaz.

Yani model neyin var olduğunu görür ama baytları görmez. *İsim ucuz, içerik pahalı*. Bizim manifest desenimiz buradan.

Bedeli: ADK bir kütüphane, tam bir platform değil; izolasyonu siz kuruyorsunuz.

1.5 · AWS — "mount et, IAM ile daralt"

Tez: Depo senin hesabında; erişimi IAM rolüyle daralt.

Akış: S3 Files ya da EFS, platform tarafından `/mnt/<ad>` altına mount ediliyor. Çift yönlü senkron. Kod düz dosya yazıyor.

Anahtar: Sandbox'ta **var** ama IAM rolü dar.

Oturum ömrü: 15 dakika – 8 saat.

Bedeli: Yönetilen bir artifact deposu yok, kayıt defteri yok. Denetim CloudTrail'de duruyor ama o bir registry değil — "kim ne zaman ne yazdı" var, "bu dosya neyden türedi" yok.

1.6 · Microsoft — "havuzdan al, işi bitince yok et"

Tez: Sandbox ucuz olsun; havuzdan milisaniyede gelsin, soğuma süresinde yok edilsin.

Akış: Kod `/mnt/data` altına yazar, `files` yönetim API'siyle dışarı alınır (128 MB sınır). Oturum bitince **her şey ölür**.

İzolasyon: Hyper-V — donanım sanallaştırma, güçlü.

Bedeli: Kalıcı depo **yok**. Oturumla birlikte gidiyor. Kayıt defteri yok.

1.7 · Red Hat / Kubeflow Pipelines — "DAG yazılıdır"

Tez: Adımlar önceden bellidir; her adımın girdisi ve çıktısı YAML/DSL'de tanımlıdır. Keşif diye bir şey yok.

Akış:

```
1 DAG'da yazıyor: bu adımın girdisi şu artifact'in .uri'si
2 kfp-driver (init container) o .uri'yi MLMD'den ÇÖZER
3 kfp-launcher dosyayı indirir, .path'e koyar
4 kullanıcı kodu BAŞLAR – dosya zaten oradadır
5 kod .path'e yazar
6 launcher onu .uri'ye kopyalar, MLMD'ye künye düşer
```

Kayıt defteri: MLMD — tip, soy, pipeline_root. Soyu DECLARED_INPUT / DECLARED_OUTPUT olaylarından çıkarıyor: **beyan edilen** girdi ebeveyn sayılıyor, kodun gerçekten okuyup okumadığına bakılmıyor.

Bedeli — ve bizim ondan ayrıldığımız yer: kfp-launcher kullanıcı koduyla aynı container'da çalışıyor. Yani S3 anahtarı kodun ulaşabileceği yerde. KFP için sorun değil — orada kodu insan yazıyor. Bizde LLM yazıyor.

1.8 · Argo Workflows — yerleşimi doğru yapan

Tez: Aktarımı kullanıcı container'ının **dışına** çıkar.

Akış:

```
init container    → beyan edilen girdileri indirir, kod başlamadan
kullanıcı kodu    → düz dosya okur/yazar
wait sidecar      → çıktıları yükler
```

argoexec kullanıcının imajından **ayrı bir imaj**. Anahtar orada, kodun ulaşamayacağı yerde.

Argo bunu deneyerek buldu: dört farklı yerleşim denemişler (docker, kubelet, k8sapi, pns) ve v3.4'te hepsini kaldırmışlar — docker için gerekçe: **"breaks security completely"**.

Bedeli: Kayıt defteri **yok**. Adres YAML'da sabit; "bu adda ne var" diye soramazsınız.

1.9 · Databricks — "denetimli mount"

Tez: Mount et, ama sürücünün içine yetkilendirme koy.

Akış: Kod /Volumes/<katalog>/<şema>/<volume> altına POSIX/FUSE ile yazıyor. Sürücü Unity Catalog'a soruyor: bu kimlik bu yola yazabilir mi?

Kayıt defteri: Unity Catalog + MLflow — bu listedeki en olgun kombinasyon.

İzolasyon: Lakeguard — Spark Connect + container sandbox + egress izolasyonu.

Bedeli: Bütün platformu gerektiriyor. Ve kendi dokümanları şunu diyor: **"Customers are responsible for running only trusted code."** Yani güvenilmeyen kod için tasarlanmamış.

En yakın rakibimiz bu — mount edip yine de kayıt defterini ayakta tutan tek ürün.

1.10 · Devin — "artifact git'tir"

Tez: Çıktı bir dosya değil, bir **pull request**.

Akış: microVM + hipervizör snapshot. RAM, süreçler, disk — hepsi donduruluyor. Süresiz uykuya geçiyor, uyandırılıyor. Artifact deposu **yok** çünkü gerek yok: iş bitince git'e commit atıyor.

Bedeli: Yalnızca kod üreten işler için çalışıyor. Bir parquet, bir grafik, bir PDF — bunların git'te işi yok.

1.11 · E2B, Modal, Daytona, Vercel, Fly.io — sandbox satanlar

Tez: Biz sandbox'ı veriyoruz, depoyu sen getir.

Akış: Senin bucket'ın mount ediliyor (ya da Fly.io'da blok cihaz). Kod düz dosya yazıyor.

Anahtar: Çoğunda sandbox'ın **içinde**.

Bedeli: Kayıt defteri yok, soy yok, TTL yok. Ve anahtar içeride olduğu için kod ele geçerse bütün bucket gider.

1.12 · Biz — "artifact'i sakla, container'ı değil"

Tez: Container her seferinde yeni doğsun; kalıcılık **artifact tarafında** olsun.

Akış:

```
1 model kod yazar ve ne okuyacağını BEYAN eder: inputs=["ozet.json"]

2 pod doğar — İÇİNDE İKİ CONTAINER:

    artifact-sidecar ← S3 anahtarı BURADA

    sandbox          ← LLM'in kodu, hiçbir sır yok

3 sidecar beyan edilen dosyaları /output'a koyar

4 sidecar bir işaret dosyası bırakır: "hazırım"

5 sandbox başlar — düz Python, hiçbir ağ çağrısı yok

6 kod biter; sidecar SIGTERM alır ve /output'a BAKAR

7 ne varsa MinIO'ya bayt, SQLite'a künye

8 pod silinir
```

Kayıt defteri: SQLite — `artifact_id`, tip, boyut, **soy**, hash, alias, TTL.

Keşif: İsimler prompt'a enjekte ediliyor (ADK deseni) ve `/output` gerçek dosyalar içeriyor, `os.listdir` doğruyu söylüyor.

Bedeli: İzolasyon bu listedeki **en zayıfı** — düz container, Kata yok.

Bölüm 2 — Keşif: ajan neyin var olduğunu nasıl öğreniyor

Bölüm 1 "bayt nereye gidiyor" sorusunu anlattı. Bu bölüm daha zor olanı: **ajan, depoda ne olduğunu nasıl biliyor?**

Pipeline dünyasında bu soru hiç sorulmuyor — DAG'ı insan yazıyor. Ajan dünyasında sorulmak zorunda, çünkü ajan da o an karar veriyor.

Yedi cevap var.

2.1 · Desen 1 — Sadece tool tarifi

Modele bir `list_artifacts()` tool'u verirsiniz; tarifini okur, **çağırmayı seçmesi** gerekir.

Sorun: yumuşak garanti. Model çağırmayı unutursa depodaki veriyi yeniden üretir — ve bunu sessizce yapar. 2026-09-06'ya kadar bizdeki hâl buydu; 40 saniyelik iş boşuna tekrarlanıyordu.

2.2 · Desen 2 — Referans context'e kendiliğinden düşer · Anthropic

Model bir dosya ürettiğinde `file_id` **tool sonucunun içinde** dönüyor ve konuşmada kalıyor.

```
model kod yazar → dosya üretir

platform dizine bakar, Files API'ye kaydeder

tool sonucu: {"file_id": "file_abc123", "filename": "rapor.pdf"}

↓

bu satır artık KONUŞMANIN içinde
```

Aramaya gerek yok — kimlik zaten orada.

Sınırı: yalnızca **bu konuşmada** üretilenler için geçerli. Üç ay önceki başka bir konuşmadaki dosyayı bulmanın yolu yok; o `file_id` bu konuşmada hiç geçmedi.

2.3 · Desen 3 — Dosya sistemi + 1s · Anthropic, OpenAI, Cloudflare, Google

Sandbox yaşıyorsa model diske bakar: `os.listdir("/mnt/data")`.

Bu desenin çalışması **tek bir şarta** bağlı: sandbox'ın yaşaması.

Ürün	Pencere
Anthropic	30 gün
Google Agent Engine	14 gün
OpenAI	20 dakika
Cloudflare	mount edilmiş bucket — süresiz
BİZ	~4 saniye

Bizde tek başına yetmez. Sandbox'ımız 4 saniye yaşıyor ve `/output` her seferinde boş doğuyor — ölçüldü. Bu bir eksiklik değil bilinçli tercih, ama bedeli keşfi başka bir yerden çözmek zorunda olmamız.

2.4 · Desen 4 — İsimler prompt'a enjekte · Google ADK

Listedeki **birinci sınıf** cevap. `LoadArtifactsTool` iki iş birden yapıyor:

"LoadArtifactsTool **lists available artifacts in the model instructions**. When the model calls the `load_artifacts` tool, ADK **temporarily appends** the selected artifact contents to that request."

Üç kuralı var:

1. **İsimler HER ZAMAN talimatlarda** — ucuz, model unutamaz
2. **İçerik TALEP ÜZERİNE** — model isteyince
3. **İçerik geçmişe KALICI yazılmaz** — sonraki turda tekrar istemeli

Birinci kural, desen 1'in yumuşak garantisini **sert garantiye** çeviriyor: model artık "çağırmayı seçmek" zorunda değil, isimler zaten gözünün önünde.

Üçüncüsü ince: bir kez yüklenen 50 MB'lık tablo sonraki her turda context'te taşınmıyor.

Bizim manifestimiz birebir bu.

Bir incelik daha: ADK'da `list_artifact_keys()` gibi metodlar **LLM'e tool olarak sunulmuyor**.

"The primary way you interact with artifacts... is through methods provided by the `CallbackContext` and `ToolContext` objects."

Yani geliştirici API'si; model çağırıyor.

2.5 · Desen 5 — Semantik arama · Llama Stack (ama başka bir şeyin araması)

file_search tool'u vector store üzerinde çalışıyor:

"...particularly useful for retrieval-augmented generation (RAG) workflows."

Files API'nin kendi tarifi: *"manages file uploads for use in embedding and retrieval workflows."*

Yani **ingest edilmiş belgeler** üzerinde anlam araması. "Ajanın ürettiği ve sonra geri aldığı artifact" diye bir kavram dokümanda hiç geçmiyor. Tabloda duruyor çünkü bir arama mekanizması — ama farklı bir problemi çözüyor.

2.6 · Desen 6 — Kayıt defterine sorgu · MLMD (ve 2026-09-08'den beri biz)

MLMD: ListOptions(filter_query="name = 'rapor.pdf' AND type = 'system.Dataset'")

BİZ: ?name=rapor.pdf&type=system.Dataset&q=rapor

Süzülmüş **künye** listesi; bayt yok.

Neden eklendi — ölçüldü: manifest her model çağrısında context'e girdiği için 40 satırla kırılmak zorunda. Depoda **57 ayrı ad** var → **17'si modele tamamen görünmez**. "Geçen ay ürettiğim raporu bul" isteği, dosya penceresinin dışında kaldıysa karşılıksız kalıyordu.

Manifesti büyütme yanlış çözüm: ucuz olması gereken şey pahalılaştırdı.

Canlı kanıt (taze oturum, manifest penceresinin dışından):

model: "listede 'kopya' geçen 4 dosya gördüm,
kesin sayı için artifact_ara kullanacağım"
arama: 24 eşleşme
sonra: birini beyanla okuyup içeriğini getirdi

2.7 · Desen — · Keşif YOK · KFP, Argo, Airflow, Tekton

Soru hiç sorulmuyor:

DAG'ı insan önceden yazar

↓

5. adımın girdisi BAĞLANMIŞ

↓

driver .uri'yi çözer, launcher .path'e indirir

↓

container doğduğunda dosya ZATEN ORADA

Keşif yerine **statik bağlama**. Karar veren bir ajan olmadığı için keşfe ihtiyaç da yok.

2.8 · Dört büyük ürünün yeri — asıl mesele

Anthropic, OpenAI, Cloudflare ve Google **aynı kutuda değiller**.

Ürün	Deseni	Tezi	Kayıt defteri
Anthropic	2 + 3	container'ı sakla — 30 gün, 5 dk'da checkpoint	file_id; soy/alias/arama yok
OpenAI	3	çalışma alanı ≠ kalıcı durum	dosya kimlikleri; kayıt defteri yok
Cloudflare	3	kod yaz, tool çağırma	yok ve olamaz — mount, araya girecek yer yok
Google ADK	4	isim ucuz, içerik pahalı	ad + sürüm

Üç ayrıntı bu tabloyu okumayı değiştiriyor:

Anthropic iki kanalı birden kullanıyor. file_id konuşmada kalıyor (desen 2) ve container 30 gün yaşadığı için ls de çalışıyor (desen 3).

OpenAI'de pencere üç büyüklük mertebesi dar. Desen 3 sandbox'ın yaşamasına bağlı; 20 dakika hareketsizlikte container gidiyor. Anthropic'in 30 günüyle kıyaslanamaz.

Cloudflare'de dosya sistemi container'ın diski DEĞİL. Doğrudan R2 bucket'ı, mount edilmiş. Sonucu: kayıt defteri yok ve olamaz — write() ile bucket arasında hiçbir katman yok.

Google'ı tek satıra sıkıştırmak hata: Agent Engine keşfi *atılıyor* (sandbox 14 gün yaşıyor), GKE Agent Sandbox artifact kavramını hiç tanımıyor, ADK ise problemi *çözen* tek birinci sınıf mekanizma.

2.9 · Yedi desen tek tabloda

#	Desen	Nasıl	Kim
---	-------	-------	-----

1	Sadece tool tarifi	Model çağırmayı seçmek zorunda	(2026-09-06'ya kadar biz)
2	Referans context'e düşer	file_id tool sonucunda döner	Anthropic
3	Dosya sistemi + ls	Sandbox yaşıyorsa model bakar	Anthropic, OpenAI, Cloudflare, Google
4	İsimler prompt'a enjekte	İsimler talimatlarda, içerik talep üzerine	Google ADK
5	Semantik arama	Vector store'da file_search	Llama Stack (RAG)
6	Kayıt defterine sorgu	filter_query ile süzölmüş liste	MLMD
—	Keşif YOK	DAG statik, girdi bağlanmış	KFP, Argo, Airflow, Tekton
3+4+6	Üçü birden	manifest promptta · /output kod başlamadan dolu · ?name= ?type= ?q=	BİZ

Üçünü birden yapan tek yer biziz — ama bu bir övünme değil, zorunluluk:

- **Desen 3** çalışıyor çünkü sidecar /output'u kod başlamadan dolduruyor
- **Desen 4** gerekli çünkü sandbox 4 saniye yaşıyor; ls tek başına yetmez
- **Desen 6** gerekli çünkü 57 ad var, manifest 40 alıyor

Diğerlerinin birer desenle idare etmesinin sebebi sandbox'larının yaşaması. Bizimki ölüyor — o yüzden üç kanal gerekiyor.

Bölüm 3 — Neyi alacağımızı nasıl belirliyoruz

Tek bir zincir; üç soru, sırayla.

3.1 · Soru 1 — Ne var? (keşif)

A · Manifest — her turda, sormadan. Sistem mesajının içine otomatik yazılıyor:

BU OTURUMDA ÜRETİLENLER

/output/ozet.json

(Dataset, 5723 bayt)

/output/dagilim.png (Artifact, 32888 bayt)

BAŞKA ÇALIŞTIRMALARDAN — bu oturumun işi DEĞİL

inputs=["wf-abc123/rapor.pdf"] (Artifact, 8170 bayt)

Sadece isimler. Bayt yok.

İki grup bilerek ayrı. Sebebi bir arıza: 2026-09-06'da liste düzdü ve ajan başka bir çalıştırmanın aynı adlı dosyasını kendi işi sanıp yanlış sayı verdi. Cevap **sessizce** yanılttı; hiçbir yerde hata yoktu.

B · Arama — model isteyince. Manifest en yeni 40 ismi alıyor; depoda 57 ad varsa 17'si görünmüyor. Model eksik olduğunu anlarsa `artifact_ara` çağırıyor.

3.2 · Soru 2 — Hangisini istiyorum? (beyan)

```
run_ptc_code(kod, inputs=["ozet.json"])
```

Bu bir çağrı değil. Hiçbir şey indirilmiyor, hiçbir yere bağlanılmıyor — sadece bir liste. Kod henüz çalışmadı bile.

3.3 · Soru 3 — Nereye düşecek? (yerleştirme)

Beyanın **yazılış biçimi** dosyanın nereye konacağını belirliyor:

Beyan	Düştüğü yer	Anlamı
ozet.json	/output/ozet.json	bu çalıştırmanın kendi çıktısı
wf-abc123/ozet.json	/artifacts/wf-abc123/ozet.json	başka bir çalıştırmanın
rapor.pdf@onaylanmıs	/artifacts/_alias/rapor.pdf	sabitlenmiş sürüm

Manifest zaten kopyalanacak biçimde yazıyor; model satırı olduğu gibi alıyor.

3.4 · Sonra ne oluyor

- sidecar beyanı okur
- dosyaları MinIO'dan indirip yerine koyar

```
3 "hazırım" der (paylaşılan diskte bir işaret dosyası)
4 sandbox başlar
5 kod açar: json.load(open("/output/ozet.json"))
```

Kod açısından bu **sıradan bir dosya okuması**. Artifact diye bir kavramdan haberi yok.

3.5 · /output ile depo aynı şey değil

Boyut	/output	Artifact deposu
Nedir	Pod'un içindeki boş disk (emptyDir)	MinIO + kayıt defteri
Ömrü	Pod'la ölür (~4 sn)	Kalıcı (TTL'e kadar)
İçinde ne var	Sidecar'ın koyduğu + kodun yazdığı	Her şey

/output bir **kopya alanı**, depo değil. Taze bir oturumda **boş** — ölçüldü.

3.6 · Kritik nokta

Seçimi model yapıyor, ama seçim kod çalışmadan önce bitiyor. Kod çalışırken "şunu da getir" diyemiyor.

Kısıt gibi duruyor; üç şey kazandırıyor:

- Sandbox'ın **hiçbir ağ çağırısı yok** — anahtar da adres de orada değil
- Soy ağacı** kuruluyor: beyan edilen girdi = ebeveyn
- Kod basitleşiyor: özel API yok, düz `open()`

KFP ve Argo tam olarak böyle yapıyor. Tek fark: onlarda beyanı **insan** yazıyor (YAML'da), bizde **model** yazıyor.

Bölüm 4 — Nicelik karşılaştırması

4.1 · Sandbox ne kadar yaşıyor

Model	Kim	Süre
-------	-----	------

Her çağrıda yeni, saklanmaz	BİZ	~4,1 sn
Efemer + cooldown'da yok edilir	Microsoft	havuzdan ms
Süre sınırlı oturum	AWS	15 dk – 8 saat
Hareketsizlikte ölür	OpenAI	20 dakika
Adlandırılmış, uzun ömürlü	Google Agent Engine	14 gün
Varsayılan yeni, id ile canlanır	Anthropic	30 gün (5 dk'da checkpoint)
SSH'lı ev dizini	Databricks Sandbox (beta)	100 GB, oturumlar arası
Hipervizör snapshot	Devin	süresiz

Aralık dört büyüklük mertebesi: 4 saniye ile 30 gün.

Ama bu sayılar aynı şeyi ölçmüyor. Anthropic *container*'ı saklıyor, biz *artifact*'i. Bizim 4,1 saniyemiz bir eksiklik değil, tercih: kalıcılığı container'dan aldık, depoya koyduk.

4.2 • Baytlar nerede, ne kadar yaşıyor

Ürün	Depo ürünü	Sandbox'taki yol	Baytların ömrü
Anthropic	belgelenmemiş	container diski (5 GiB)	30 gün
OpenAI	belgelenmemiş	container diski	20 dk hareketsizlik
Cloudflare	R2 / S3 / GCS	mount noktası	bucket'ın ömrü
Google (ADK)	GCS / bellek / yerel disk	ArtifactService API	deponun ömrü
AWS	S3 Files / EFS (senin hesabın)	/mnt/<ad>	bucket / EFS
Microsoft	yok	/mnt/data	oturumla ölür
Red Hat / KFP	S3 / GCS / MinIO	.path (launcher kopyalar)	pipeline_root
Databricks	Unity Catalog Volumes	/Volumes/...	volume'ün ömrü

Devin	yok — git	makine snapshot'ı	süresiz
E2B / Daytona / Vercel	senin bucket'ın	mount noktası	bucket'ın ömrü
Fly.io	S3-uyumlu	kök FS (100 GB)	Sprite'ın ömrü
BİZ	MinIO / ODF / harici S3	/output + /artifacts/<wf>	TTL + reaper

4.3 • Kaynak sınırları

Ürün	Bellek	Disk	CPU	Süre sınırı
Anthropic	5 GiB	5 GiB	1	90 sn / REPL hücresi
Microsoft	—	—	—	128 MB dosya sınırı
Fly.io	—	100 GB	—	—
Databricks	—	100 GB	—	—
BİZ	1 GiB	node diski	16	90 sn (activeDeadlineSeconds)
BİZ — artifact	—	100 MiB / dosya	—	—

Bizim 1 GiB'ımız Anthropic'in 5 GiB'ının beşte biri. Bu, gerçek bir kısıt: büyük bir veri seti sığmaz.

4.4 • Kayıt defteri — var mı, ne tutuyor

VAR	Ne tutuyor	YOK
BİZ	artifact_id, tip, soy, TTL, hash, alias	E2B, Modal, Daytona, Vercel
Anthropic	Files API — file_id	Cloudflare, Fly.io, Microsoft
Red Hat / KFP	MLMD — tip, soy, pipeline_root	AWS (CloudTrail denetim, registry değil)
Google ADK	artifact adı + sürüm	Devin (çıktı git'te), Cerebras
Databricks	Unity Catalog + MLflow	

On bir üründen beşinde kayıt defteri hiç yok. Ve hepsi mount eden aile.

Dokümanları "persistent data access" diyor; hiçbiri **"artifact"** demiyor. Fark önemli: bir S3 anahtarı bir dosyayı bulur, ama o dosyanın neyden türediğini, hangi sürümün onaylı olduğunu, ne zaman sileceğinizi söylemez.

4.5 · Bizim ölçülmüş sayılarımız

Ölçüm	Değer
Bir çalıştırma	~4,1 sn (okuyan çalıştırma 3,13 sn)
Birim/entegrasyon testi	225
Canlı kabul kontrolü	52/52 (proxy) · 53/53 (direct)
Depoda	313 künye · 84 çalıştırma
MinIO'da	281 nesne
Fark (dedup)	32 künye aynı baytı gösteriyor
Sandbox'ın ağ çağırısı	0

Bölüm 5 — Güvenlik karşılaştırması

5.1 · İzolasyon — kod nerede çalışıyor

Yöntem	Ne demek	Kim	Güç
Container (runc)	Kernel paylaşılır, sadece namespace ayırır	BİZ	En zayıf
V8 isolate	JS motoru içinde bölge	Cloudflare	Dar ama sıkı
gVisor	Araya sahte kernel girer	Google GKE	Orta
Lakeguard	Spark Connect + container + egress	Databricks	Orta-güçlü

izolasyonu

Kata / microVM	Pod başına kendi kernel'i	Red Hat önerisi, E2B, Vercel, Devin	Güçlü
Hyper-V	Donanım sanallaştırma	Microsoft	Güçlü

Bu listede en zayıf olan biziz. Red Hat, AI-üretimi kod için açıkça Kata öneriyor. İyi haber: kod değişikliği gerektirmiyor, node seviyesinde bir önkoşul.

5.2 · Anahtar kimde — asıl soru bu

Kod ele geçerse ne kaybedersiniz? Cevap, anahtarın nerede durduğuna bağlı.

Yaklaşım	Kim	Anahtar sandbox'ta	Kod ele geçerse
Mount, anahtar içeride	E2B, Daytona, Vercel (düz)	EVET	Bütün bucket
SDK, anahtar pod'da	Red Hat / KFP, AWS	EVET (dar)	Rolün izin verdiği her şey
Mount, anahtar dışarıda	Cloudflare, Vercel (proxy), Daytona (Volumes)	Hayır	Proxy'nin izin verdiği
Denetimli mount	Databricks	Hayır	Sürücünün izin verdiği
API, anahtar hiç yok	Anthropic, OpenAI, BİZ	HAYIR	Hiçbir şey

Bu, mimarideki en önemli tek karar. Bizde anahtar **sidecar container'ında**; LLM'in kodu onunla ortam paylaşıyor. Ölçüldü:

sandbox'tan bakıldığında:

S3 kimlik bilgisi	[]
S3 SDK kurulu	hayır
kapsam jetonu	[]
servis adresi	yok
MinIO'ya IP ile	TimeoutError
internet	ConnectionError

5.3 · Baytı kim taşıyor — dört aile

Bu, "araya denetim koyabilir misin" sorusunun cevabı.

Aile	Model	Araya girecek yer
A — Mount	<code>write()</code> → FUSE/NFS → bucket	Yok
B — Sarmalayıcı	launcher, kullanıcı koduyla aynı container	Var ama kod onu atlayabilir
C — Sınır	kod dizine yazar → ayrı güven alanı taşır	Var, kod erişemez
D — Denetimli mount	mount, ama sürücü yetkilendirme uyguluyor	Sürücünün içinde

Ürün	Yükleyici nerede	Aile
Argo Workflows	<code>wait</code> sidecar (ayrı container)	C
Tekton	entrypoint + enjekte edilmiş sidecar	C
Anthropic / OpenAI / Microsoft	platform, dışarıdan hasat ediyor	C
BİZ	<code>artifact-sidecar</code> (ayrı container)	C
Databricks	FUSE sürücüsü (compute plane)	D
KFP / OpenShift AI	<code>kfp-launcher</code> , main container	B
AWS	yükleyici yok — platform NFS mount	A
E2B / Daytona / Vercel / Modal / Fly	yükleyici yok — kernel/FUSE	A

B ailesinin sorunu: launcher kodla aynı container'da. Kullanıcı kodu onu atlayıp doğrudan S3'e gidebilir. KFP için kabul edilebilir çünkü orada kodu insan yazıyor. Bizde LLM yazıyor — bu yüzden C ailesini seçtik.

5.4 · Ağ duruşu

Duruş	Kim
-------	-----

Tamamen kapalı	Anthropic — "Completely disabled for security"
Varsayılan reddet + allowlist	BİZ, Red Hat / OpenShift, Google GKE
UDF egress izolasyonu	Databricks (Lakeguard)
Opsiyonel kontroller	Microsoft
Açık, yapılandırılabilir	AWS ("network modes")

Bizde ayrıca **iki servis bilerek ayrı**:

Tool Gateway	→	internete çıkar,	depoya çıkamaz
Artifact Service	→	depoya çıkar,	internete çıkamaz

Birinin ele geçirilmesi ikisini birden vermiyor.

5.5 · Kritik ayrım: kim güvenilmeyen kod varsayıyor

Bu tablo, ürünlerin **kendi dokümanlarının ne dediğine** dayanıyor.

Ürün	Kodu güvenilir mi varsayıyor	Kanıt
Databricks	Evet	"Customers are responsible for running only trusted code"
KFP / OpenShift AI	Evet	Launcher kullanıcı container'ında; anahtar kodun yanında
Anthropic	Hayır	Ağ tamamen kapalı, anahtar sandbox'ta yok
Cloudflare	Hayır	V8 isolate, anahtar proxy'de
Google GKE	Hayır	gVisor + varsayılan reddet
E2B / Daytona / Vercel	Kısmen	İzolasyon güçlü ama anahtar içeride
BİZ	Hayır	Anahtar sidecar'da, ağ kapalı — ama izolasyon zayıf

Buradaki asıl gözlem şu: **güvenilmeyen kod varsayan ürünler, kayıt defterini de koruyabilenler**. İkisi aynı mimari kararın sonucu — yazma yolunu bir bileşenden geçirmek.

5.6 · Bizim açıklarımız — dürüst liste

Açık	Durum	Etki
İzolasyon	Düz container, Kata yok	En büyük açık. Kod değişikliği değil, kurulum
Auth	Yok	Jetonu üretebilen tenant'ın tümünü okur
Metadata DB	SQLite	Tek replika
Soy imzasız	Kayıt defterine yazabilen değiştirebilir	Tekton Chains bunu çözüyor, bizde yok
SIGKILL	OOM/deadline'da süpürme çalışmaz	O ana kadarki iş kaybolur (<i>Argo'da da aynı</i>)
Büyük dosya	100 MiB servis / 1 GiB pod	5 GB çalışmaz
Dosya yükleme	Kullanıcı sohbete dosya atamıyor	Anthropic/OpenAI'de manşet özellik
Paket seti	8 paket, pip install yok (ağ kapalı)	scikit-learn, scipy yok
Gerçek OBC/ODF	Test edilmedi	ODF kapsam dışı

Bölüm 6 — Sonuç

6.1 · Üç aile, üç farklı problem

Karşılaştırma yaparken en sık yapılan hata, bu üçünü aynı kefeye koymak:

Aile	Kimler	Çözdüğü problem
Sohbet platformları	Anthropic, OpenAI, Microsoft	"Konuşma devam etsin"
Sandbox satıcıları	E2B, Modal, Daytona, Vercel, Fly	"Kod güvenli bir yerde çalışsın"

Pipeline sistemleri

KFP, Argo, Tekton, Databricks

"Üretilen şey kurumsal bir varlık olsun"

Biz üçüncü ailedeyiz ama birinci ailenin girdisiyle çalışıyoruz: kodu insan değil LLM yazıyor.

Bu birleşim listede başka kimsede yok. Pipeline sistemleri kodu güvenilir varsayıyor; sohbet platformlarında ise soy ve sürüm diye bir kavram yok.

6.2 · Ne alıp nereden aldık

Parça	Kimden
İki container, sidecar taşır	Argo Workflows
Girdiyi kod başlamadan yerleştir	Argo `init` / KFP `driver` + `launcher`
Girdi beyanı · beyandan soy	Argo `inputs.artifacts` · MLMD `DECLARED_INPUT`
Çalıştırma başına anahtar yolu · kayıt defteri	KFP `pipeline_root` · MLMD
Künye süzgeci · sürüm alias'ı	MLMD `filter_query` · MLflow Model Registry
İsimler prompt'a enjekte	Google ADK `LoadArtifactsTool`
/output süpürme	Anthropic, OpenAI
Sandbox'ta sıfır ağ çağırısı	KFP
Hata sinyali · kırpmalı · "ne yapmalı"	SWE-agent · smolagents · Codex
İzolasyon	Kimse — bizimki daha zayıf

Emsalsiz desen kalmadı. Dört şey icat ettik, dördünü de attık — ve şunu ölçtük:

Hataların hepsi bizim icat ettiğimiz yerlerde çıktı; kopyaladığımız hiçbir parçadan çıkmadı.

6.3 · MLMD'yi kullanıyor muyuz — hayır, desenini alıyoruz

Sık gelen soru. Ayırım net:

MLMD'den ALDIK	MLMD'den ALMADIK
Event . DECLARED_INPUT / DECLARED_OUTPUT soy semantiği	MLMD sunucusunun kendisi

ListOptions(filter_query=...) süzgeci	gRPC API'si
Tipli artifact (system.Dataset vb.)	Şeması
pipeline_root/<run-id>/ anahtar düzeni	—

Yerine SQLite ve kendi şemamız:

```
artifact_id · name · workflow_id · node_id · run_id

content_hash · content_type · size_bytes · storage_uri

parents[] · owner · created_at · ttl_seconds · alias
```

Gerekçe uydurma değil, MLMD'nin sahibinin kendi hamlesi:

Red Hat, **OpenShift AI 2.23'te Model Registry'den MLMD sunucusunu kaldırıp kendi şemasına geçti** — gerekçe *"mimariyi basitleştirmek, uzun vadeli sürdürülebilirlik."*

Yani MLMD'yi kurmamak, MLMD'yi en çok kullanan platformun gittiği yönle **aynı** yön. SQLite tercihi de öyle: Red Hat'ın çizgisi PostgreSQL üretim / SQLite geliştirme. open_postgres() yazılı ve bekliyor; SQL taşınabilir yazıldı (yalnızca TEXT/BIGINT, ISO-8601 zaman, JSON parents).

Özet: MLMD'nin mentalitesi var, implementasyonu yok.

6.4 · "Agentic PTC için SOTA yaklaşım bu mu?"

Hayır — çünkü ortada bir SOTA yok.

Sebebi yapısal: bu birleşim piyasada mevcut değil.

PIPELINE SİSTEMLERİ		AJAN PLATFORMLARI	
(KFP, Argo, MLMD)		(Anthropic, OpenAI)	
artifact + soy + tip	✓	artifact + soy + tip	✗
karar veren ajan	✗	karar veren ajan	✓
keşif	✗	keşif	✓
(DAG'ı insan yazar)		(ama soy yok)	

Pipeline sistemlerinde keşif **yok** çünkü gerek yok — karar veren bir ajan yok. Ajan platformlarında soy **yok** çünkü problemleri o değil: *"sohbet devam etsin"* diyorlar, *"bu dosya neyden türedi"* demiyorlar.

Databricks en yakını, ama kendi dokümanı *"Customers are responsible for running only trusted code"* diyor —

yani bizim girdimiz için tasarlanmamış.

İddia edebileceğimiz şey

SOTA değil, ama şu doğru: **her parçanın tek tek emsali var ve emsalsiz olan hiçbir parça kalmadı.** §6.2'deki tablo bunun listesi.

Dört şey icat ettik, dördünü de attık. Ölçülmüş sonucu:

Hataların hepsi bizim icat ettiğimiz yerlerde çıktı; kopyaladığımız hiçbir parçadan çıkmadı.

SOTA iddiasını zayıflatan üç şey — dürüstçe

#	Zayıflık	Durum
1	İzolasyon listedeki en zayıfı	Düz container, Kata yok. Red Hat AI-üretimi kod için açıkça Kata öneriyor. Mimari kusur değil, kurulum eksikliği — ama duruyor
2	Arama tool'unun emsali zayıf	ADK <code>list_artifact_keys()</code> 'i modele açmıyor ; Llama Stack'in <code>file_search</code> 'ü RAG. Sorgu MLMD'nin, kural ADK'nın, ama <i>"modelin çağırabileceği artifact araması"</i> kombinasyonu bize ait
3	Ölçek denenmedi	313 artifact, tek node, SQLite tek replika. 100 bin artifact'te ne olacağını bilmiyoruz

İkincisinin savunması var ama emsali yok: manifest **sert** kanal (her turda, model unutamaz), arama **yumuşak** ikinci kanal (yalnızca pencerenin dışı için). ADK yalnızca birincisini yapıyor — ve haklı bir sebeple: yumuşak kanal tek başına 2026-09-06'daki arızayı üretir. Bizim eklememizin savunması, onun tek kanal **olmaması**.

Kısa cevap

SOTA bir yaklaşım değil — SOTA'sı olmayan bir boşlukta, kanıtlanmış parçalardan kurulmuş bir birleşim.

Bu, "en iyisini yaptık" demekten daha savunulabilir bir iddia: her parçayı çalıştığı kanıtlanmış bir yerden aldık, ve neyi nereden aldığımız yazılı.

6.5 • Tek cümlelik karşılaştırma

Onlar container'ı saklıyor, biz artifact'i. Onların çözdüğü problem *"sohbet devam etsin"*; bizimki *"üretilen şey kurumsal bir varlık olsun — kimin ürettiği, neden türediği, hangi sürümün onaylı olduğu belli olsun."* Soy, alias ve arama onlarda yok — çünkü onların problemi o değil.